

UNIVERSITY OF
Waterloo



Department of Mechanical and Mechatronics Engineering

**The Design of a Testable Hybrid
Rocket Engine Using Software
Analysis**

A Report Prepared For:
The University of Waterloo

Prepared By:
Artem Sotnikov
9471 Yonge Street
150 University Ave W.
Richmond Hill, Ontario, L4C 0Z5

January 23, 2023

9471 Yonge Street
150 University Ave W.
Waterloo, Ontario, N2N 2N2

January 23, 2021.

Andrew Kennings
Director of Mechatronics
Department of Mechanical and Mechatronics Engineering
University of Waterloo
200 University Ave W.
Waterloo, Ontario, N2L 3G1

Dear Professor Kennings,

This report, entitled *The Design of a Testable Hybrid Rocket Engine Using Software Analysis*, was written to fulfil my WKRPT300 work term report requirements. This is my 3rd work report submission.

This project explores the conceptual design of a hybrid rocket engine. The core aspect that I worked on was the design of the software analysis packages for it, which will be the focus of the report. However, other important design decisions that I was involved in will be discussed as well. Various numerical methods and mathematical approaches were explored and applied in order to come up with the most accurate estimates of the theoretical engine's performance metrics. This in turn was of immense importance for designing the thermal and mechanical systems for the rest of the hybrid rocket engine. Other aspects that were considered as desirable upgrades was the use of lighter materials, different system ratios, and other approaches. The ultimate fruits of this project (which is still ongoing) have already significantly contributed to the development of the 2023 *Leviathan of the Sky* rocket for the Waterloo Rocketry university design team, that is to fly in an international competition in the summer of 2023.

This report was written entirely by me, and has not received any previous academic credit at this or any other institution. I would like to acknowledge the help of fellow teammates. Aaron Leszkowiat aided considerably in the theoretical background required to model the thermodynamic processes. I would also like to acknowledge the help of Carol Xu and Paniz Ojaghi, who successfully developed portions of the code under my direction. Furthermore, I would like to thank the other members of the team for assisting me in the fabrication of parts for this project. Without them, I would not have had the time to contribute to the project and also write this report.

The financing for the team, which allows us to continue operations and to bring our designs to life, is provided by several organizations within the university and external sponsors. They are ultimately what make projects like this possible. A full list of them can be seen on Waterloo Rocketry's website. I would like to thank them for their contributions.

Best Regards,

Artem Sotnikov (3B Mechatronics Engineering, Student #20845874)

TABLE OF CONTENTS

Summary	4
1.0 Introduction.....	5
2.0 Background analysis, problem statement, Considerations for the propulsions system and overall launch vehicle architecture.....	7
3.0 Directions of development	9
4.0 Design of the engine modelling software.....	12
5.0 Conclusion	21
References.....	22
Appendix A: Additional Figures.....	24
Appendix B: Code.....	28
B.1 Trilinear Interpolation	28

TABLE OF FIGURES

<u>Figure 1: Broad overview of rocket architecture (taken from [3]).....</u>	<u>5</u>
<u>Figure 2: N2O compressibility graph (provided by P. Ojaghi , September 23, 2022).....</u>	<u>13</u>
<u>Figure 3: Debug printout of a blowdown simulation.....</u>	<u>14</u>
<u>Figure 4: Complex geometry regression model (provided by P. Ojaghi on November 5, 2022)</u>	<u>15</u>
<u>Figure 5: Equation Relating Area ratio and mach number [23].....</u>	<u>16</u>
<u>Figure 6: Mach number curve as a function of area ratio [24].....</u>	<u>16</u>
<u>Figure 7: Complete simulation running at a 0.01s time step.....</u>	<u>17</u>
<u>Figure 8: Complete Simulation running at a 0.05s time step.....</u>	<u>17</u>
<u>Figure 9: Debug printout of time step (seconds on both axes) taken during development.....</u>	<u>18</u>
<u>Figure 10: Scope capture of Simulink Simulation</u>	<u>20</u>
<u>Figure 11: Reference case for airframe diameter comparison: 6 inches.....</u>	<u>24</u>
<u>Figure 12: Test case for Airframe diameter comparison: 8 inches</u>	<u>25</u>
<u>Figure 13: RPA Calculation Results (provided by Aaron Leszkowiat , October 19, 2022)</u>	<u>26</u>
<u>Figure 14: Cantera Calculation results (done on October 19th, 2022).....</u>	<u>27</u>

SUMMARY

In this project, the software analysis tools of a hybrid rocket engine were constructed. This engine was constructed for an experimental sounding rocket that is to participate in a student competition, the Intercollegiate Rocket Engineering Competition (IREC). Issues identified with the propulsion system during the previous competition cycle led the team to invest time in the development of a new engine and to optimize its architecture.

An analysis of the exact root causes for the problem is done, and the need for an rocket more flight-optimized is justified. Several approaches to performance improvement are discussed as part of conceptual design. Core requirements are identified, and the resulting tradeoffs of various optimization strategies are discussed.

Afterward, the report focuses on one optimization strategy and discusses the full development cycle of a software analysis package that enables the development of a hybrid engine to meet the initial requirements. The specific requirements of the software are explored in more detail. Various numerical schemes are outlined, and their effects on the development process are analyzed. Furthermore, various challenges encountered by the development team are brought up, and the engineering solutions found to them are described.

Lastly, the report considers a validation approach through a mathematic derivation and the use of a different software package.

1.0 INTRODUCTION

The Waterloo Rocketry Team has developed experimental sounding rockets since 2009. The stated goal of the organization, which still holds today, is to enable its members to learn engineering by tackling interesting technical project in the sphere of aerospace engineering. As a consequence, initially the rockets were quite simple, over time the team has grown and evolved to design sounding rockets of increasing complexity and technical sophistication. Furthermore, the team has increasingly pivoted to improving its performance at the Intercollegiate Rocket Engineering Competition (IREC), also referred to as the Spaceport America cup (SA cup). Performance at this competition drives the team to develop highly optimized systems on a tight schedule, reminiscent of real aerospace design. Furthermore, good performance looks particularly good on a resume when applying to jobs, as the competition is the most prestigious international event of its kind.

It is worth discussing the rough architecture of the rocket, which is fairly standardized in the current class of rockets that the team has built since 2018. The rocket is a hybrid rocket, which means that it has a solid fuel and a liquid oxidizer [1]. This is advantageous for safety purposes as the two energetic fluids are stored separately. At the same time, this avoids the complexity of a liquid engine, which has to have a more complex feed system and at least two pressurized tanks, along with a more complex startup and injection system. The propellant combination utilized by the team is Hydroxy-terminated polybutadiene (HTBP), an epoxy with a long history in aerospace [2] and Nitrous Oxide (N₂O), commonly known as ‘laughing gas’. This combination has a flat impulse curve, making it easy to use [1].

The rocket’s layout can be seen in the figure below. A large hybrid engine consisting of a combustion chamber, feed system, and oxidizer is located at the bottom. This system will be the main focus of this report. Above it, the payload of the rocket is housed, which must be a 3U CubeSat format as per the rules of the competition. Finally, a recovery section is located at the top of the rocket, which enables the rocket to be recovered safely (also per the rules of the competition). The avionics of the rocket are a suite of custom PCBs running on a Controller Area Network (CAN). The airframe is made up of custom composite fiberglass and carbon fiber parts, which includes the nosecone and fins.



FIGURE 1: BROAD OVERVIEW OF ROCKET ARCHITECTURE (TAKEN FROM [3])

The rocket is meant to achieve an apogee of 30 000 ft. Additionally, the rules of the competition require the engine to be static fired in advance to ensure safety. Combined with the logistics of transporting the rocket to competition and the cost of its development, significant design focus has to be aimed at making it as small and mass-efficient as possible.

The team had previously flown this class of rocket in 2019, where it suffered a failure of the recovery system. This led to subsequent upgrades in the recovery system, which are detailed in extensive detail in the previous work term report submitted by the author. In 2022, The vehicle had significant issues with its electronics, and large fundamental issues with its architecture due to its inefficient engine were revealed. These factors combined led to a scrub of the launch and a mediocre overall performance at the competition. The tackling of the latter problem was one of the main directions of development selected for the rocket in 2023, and which the author was able to significantly contribute to.

2.0 BACKGROUND ANALYSIS, PROBLEM STATEMENT, CONSIDERATIONS FOR THE PROPULSIONS SYSTEM AND OVERALL LAUNCH VEHICLE ARCHITECTURE

As mentioned earlier, the launch campaign of 2022 was not successful. Following this setback, the core members of the team, which included the author, performed a detailed analysis of what had wrong. Several important conclusions were reached.

A core issue identified with the propulsion system during the competition cycle was the low efficiency of the system, which required more oxidizer to achieve a set amount of impulse that what is theoretically and practically possible. The theoretical specific impulse (force divided by propellant weight consumed per second, or efficiency) of the propellant combination was over 250 s [1], while that of our engine was less than 150 s. Meanwhile, other teams at the competition could easily achieve over 180 s [4] [5]. This meant that Waterloo's system had to have a larger oxidizer tank, which significantly increased both the length and mass of the rocket. Since both are considerations for handling and transportation, it meant that the design was heavily constrained, and could not afford significant factors of safety on additional fuel or last-minute mass additions.

The direct consequence of the rigidity was the loss of performance as the system has to inevitably be modified. Final mass measurements made before the competition were higher than expected, hurting the targeted apogee before the rocket was even shipped out. Also, the weak engine meant that the rocket would possess poor off-the-rail stability, and consequently had to have a large rail. The rail used was 40 feet tall [3], and posed its own set of challenges in assembly, transportation, and raising. Finally, the low stability meant that there were punishing restrictions on maximum allowable winds during a launch, complicating an already difficult launch campaign that would be staffed by inexperienced personnel due to the impacts of the COVID-19 pandemic.

As is stated in any aerospace textbook, the rocket's engine is the central part of the design. It drives almost all aspects of the remaining design, and the success of a mission directly depends of the engine's performance. A basic equation that encapsulates these statements is Tsiolkovsky's rocket equation [1]: which states that the change in velocity of a launch vehicle (i.e. the ability to achieve a specific trajectory desired by the designers is related to the mass fraction of the vehicle, and its specific impulse. That is, the lower the dry mass of the vehicle, the more efficient it is.

$$\Delta V = C_e \ln \left(\frac{m_{start}}{m_{burnout}} \right) \quad (1)$$

The goal of the project could now be distilled to broadly improve the rocket's flight performance. By analyzing the rocket equation, it was concluded that in order to improve the performance, there were two broad solutions. Either mass could be significantly reduced, or the efficiency of the engine could be significantly increased. At this point, the engineering team began to identify key requirements that would need to be met by any solution. The primary requirements identified were as follows, in order of importance.

1. The selected design shall be safe to work with and must conform to competitions rules.
2. The selected design shall be possible to develop and test given the time, resources, and money available to the team
3. The selected design shall propel the rocket to 30 000 ft; it shall also satisfy minimum structural and stability requirements as defined by the flight dynamics team

Secondary requirements were:

1. The selected design should be as easy as possible to assemble and handle
2. The selected design should be as light, cheap, and small as possible
3. The selected design should maximize the number of points gained at the competition.
4. The selected design should be interesting to develop for the technical team such that they gain valuable experience from participating in the project.

The last point is present since as mentioned previously, the team's primary objective is to provide interesting opportunities. Furthermore, since all members of the organization are not salaries, a design that is monotonous or tedious to develop will likely not receive much interest and personnel, which would be threat to the second primary requirement (possible to develop on time and with the resources available). The rest of the points are fairly straightforward from an analytical standpoint.

The team consists of roughly fifty active members, although due to varying schedules and commitments, more than several usually do not work on a project at once. The team has roughly 10 000 CAD to develop a rocket, although services or products can sometimes be received for free or at a discount from sponsors. The timeline is one year, which is the cadence of the competition. The team also has access to a machine shop and other resources on the university campus. Furthermore, due to the student distribution at Waterloo, the personnel available to it are usually skilled engineering students that have experience in many aspects of design, including software.

Given this set of resources, the approach taken by the engineering team was to try and tackle the problem from several directions and utilize the diverse makeup of personnel available to the team.

3.0 DIRECTIONS OF DEVELOPMENT

Several specific solutions were identified at this point. The first, and perhaps most straightforward one was to simply increase the size of the propellant tank. However, this solution was rejected as it would increase the length of the rocket, which makes it more difficult to handle and stabilize in flight. Furthermore, performing calculations based on the BENDIT7 calculator [6] made it clear that increasing length would lead to increased loads on the airframe, which may be difficult to mitigate from a manufacturing standpoint.

Other approaches such as cutting down the mass of the payload were rejected as they would be associated with a decrease in the competition performance. Similarly, switching to a simpler and lighter solid propellant could not be considered due to the mundane nature of the design, which would not attract many members for its development.

A more viable approach identified would be to re-design the heaviest component of the rocket, which was the oxidizer tank. The existing iteration was made out of 6061 Aluminum. By utilizing a stronger alloy, Aluminum 7075 [7], the walls could be made thinner and the weight significantly reduced. While hollow tubes of 6" diameter could not be found online, it was considered feasible to machine the tank out of a solid round [5], which would also be advantageous as it could allow the bolts on the tank bulkheads to be oriented axially rather than radially, increasing their performance. While the 7075 would be expensive and would require significant effort to machine, calculations of the performance gains led this design to be selected for development.

Another approach related to the oxidizer tank was to change the bulkheads on it to a welded design instead of a bolted design, which would simplify assembly and save weight. While there were concerns raised that this would be difficult to sanitize prior to filling with oxidizer, conversations with an expert in the field (A. Trumpour, founder and president of Launch Canada, Summer 2022) revealed that this could be mitigated by utilizing chains to scrub the bottom of a welded tank. Furthermore, conversation with other teams at the competition led to the conclusion that this design would not be as difficult to manufacture as originally anticipated. However, ultimately the welded concept was rejected due to a lack of impact on the performance. A conceptual CAD model was created by members of the team, and the resulting mass savings were not sufficient to continue pursuing the design. Furthermore, this design would not be compatible with the 7075 oxidizer tank, as 7075 could not be conventionally welded [8] with the processes available to the team.

Yet another approach for reducing overall system mass was to change the diameter of the whole airframe from 6 inches to 8 inches. This idea came about by observing many of the other teams at the competition, which use 8 inch airframes. Theoretically, it would reduce the loads on the airframe [6]. Furthermore, it would allow the mass of the overall vehicle to be reduced by following geometrical logic; a shorter cylinder can enclose the same volume for less surface area. This approach was rejected following the completion of flight simulations done in OpenRocket software suite [9]. The 8 inch airframe would significantly increase the rocket's drag, ultimately leading to a reduction in apogee. This is demonstrated in screen captures of the simulations in Figure 11 and Figure 12 in appendix A, where the apogee is reduced by over 1000 ft. Furthermore, there would need to be a change of tooling for airframe manufacturing, and significantly larger fins, which can also be seen in the figures mentioned earlier. That would also likely in turn result in a need to change manufacturing processes for the fins to switch to a cored fin design. This is the only way to feasibly manufacture large, lightweight fins, by utilizing a lightweight foam core and a composite outer surface, increasing the rigidity of the composite [10]. This was all deemed too much effort.

Other minor weight saving approaches were pursued by the team that are beyond the scope of this report. This includes a redesign of the recovery electronics sleds, and the consideration of Al 7075 for several bulkheads. Furthermore, the injector section would be 3D printed out of metal by the university's MSAM. However, none of the approaches discussed up to this point tackled the engine's poor efficiency.

That led to the final major direction of development selected for solving the problem. The hybrid engine would be significantly re-designed to try and improve the combustion efficiency. However, this posed a large risk from an operational and timeline perspective. Since the engine had to be successfully static fired prior to competition, there was not significant time to iterate a design. A static fire is a significant undertaking that is very demanding to personnel.

For reference, executing a static fire of a engine design requires significant machining work to fabricate the engine. More work still is required to assemble the plumbing and prepare the test site. Finally, the day of the static fire requires of a crew of around ten people to move the engine and propellants to the test site. After that, they must perform leak checks, set up the data acquisition systems, and execute the firing with an established exclusion perimeter. In short, the preparation and execution of a static fire is a multi-week exercise that takes up most of the team's resources. For this reason, the timeline could only allow for a maximum of two static fires prior to the competition.

As a result, in order for engine upgrades to be considered, it was determined that this effort would be necessary to support with significant analysis. The burn duration, thrust, and other parameters would

need to be accurately estimated ahead of time to design the supporting systems in parallel to the engine. Furthermore, data analysis tools would need to be created that could accurately analyze the data from preliminary tests, such as ‘cold flows’. ‘Cold Flows’ are a test of the injection system using the comparable liquid CO₂ instead of N₂O, which is quite safe and a good way to validate the design of the injector [1] [3]. This is necessary for accurately sizing the nozzle. In the past, multiple tests of this nature were required, even for injectors that did not have a significantly different design. This was done primarily due to the presence of significant noise in the sensor readings. This time, the injector would need to be rapidly iterated, and tools were needed that could obtain the flowrate through it even from the noisy data of a single test. Due to the available software talent present on the team, this approach was also accepted. For the sake of brevity and conciseness, the remainder of the report will focus on the engine modeling efforts.

4.0 DESIGN OF THE ENGINE MODELLING SOFTWARE

Modelling hybrid engines is something that has been done, both by other teams [9] [11] and by former members of the Waterloo Rocketry Team. Resources are also available that outline the software implementation of various aspects of a hybrid engine. However, there did not already exist a single solution that met all of the following desirable criteria:

1. Written in a modern programming language
2. Capture the full dynamics of the hybrid engine including the feed system, combustion chamber, and nozzle
3. Could be easily modified
4. Were free and open-source
5. Could model advanced fuel compositions

Therefore, it was decided to develop an implementation from scratch. Since some of the previous resource utilized C++ [12], it was considered to write it in this language. However, more team members were acquainted with python, which is a more relevant language in today's software environment with more library support. Since the project would need to be done with the collaboration of other members, some of whom may not be as familiar with programming languages, the simplicity of python was also taken into account, and it was selected as the language to be used in the implementation.

The software work to be done could be broken up into several semi-independent components. For ease of project management and reference by future engineers of waterloo rocketry, git was selected to be used during development. Furthermore, that meant that separate features could be developed in parallel on various git branches. All code and development progress can be observed by viewing the public Github page for this project [13].

The first of the require features is the model of the tank's blowdown dynamics. [12] was a great resource in developing the code. However, the code snippets that it featured were highly disorganized and frustratingly written; they had to be re-done almost completely.

N₂O is self-pressurizing, which means that it can be run in a 'blowdown' scheme, being let out of its tank under its own pressure. This is contrasted to other oxidizers such as liquid oxygen, which usually require a pressurant or a pump [1]. At first, when the tank is opened, liquid exits the tank via the injector, which can be modelled using the homogenous form of the valve equation [14]:

$$Q = C_d A \left(\frac{\Delta P}{\rho} \right)^{0.5} \quad (2)$$

Furthermore, as the liquid volume is decreased, more and more of the liquid is also vaporized to maintain the vapor pressure inside the cylinder, maintaining a vapor-liquid equilibrium inside the tank. This means that there is a decrease of temperature as enthalpy is taken to vaporize the gas, which follows the following equation described in [12]:

$$\Delta T = \frac{-\Delta Q}{m_{liquid} C_{liquid}} \quad (3)$$

This process proceeds until the tank has fully emptied of liquid. After that, the tank follows an isentropic emptying process. That can be modelled using isentropic flow equations provided by thermodynamics [15]. Furthermore, gas compressibility must be considered in this final stage. Code for this latter functionality was handled by another programmer under the direction of the author. An image of an implementation test case can be seen below.

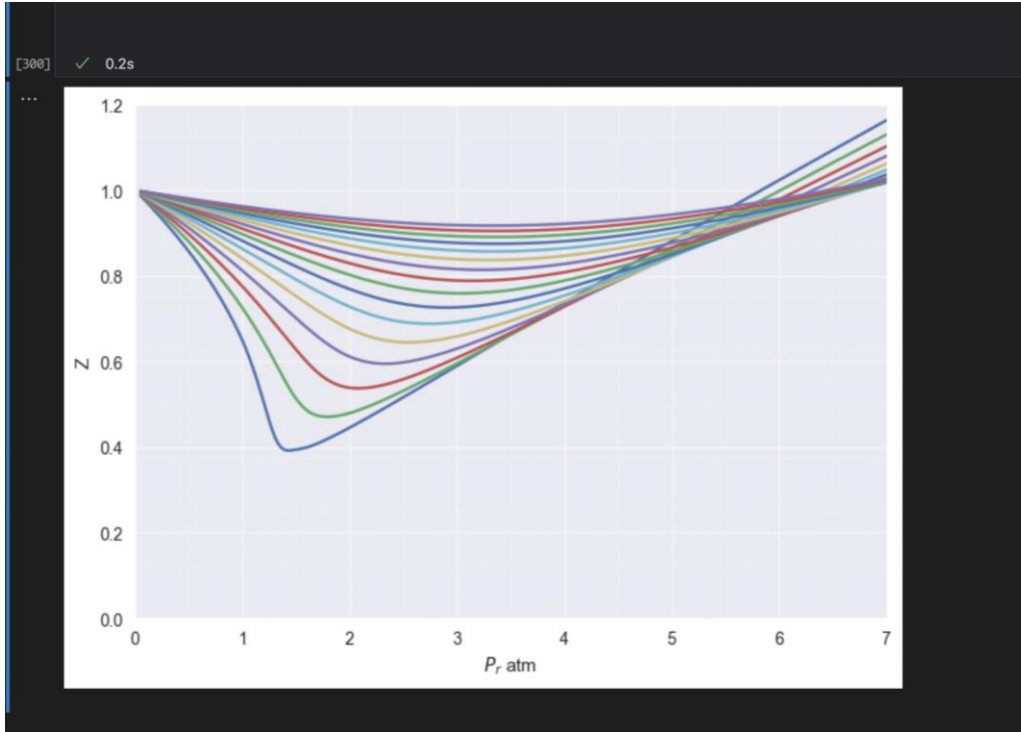


FIGURE 2: N2O COMPRESSIBILITY GRAPH (PROVIDED BY P. OJAGHI , SEPTEMBER 23, 2022)

However, the full model requires the knowledge of the pressure in the combustion chamber to accurately calculate ΔP from equation (2). However, this portion of the code was developed first, and for testing it, a simple pressure drop of 50% was used. In short:

$$\Delta P' = P_{tank} \cdot 0.5 \quad (4)$$

This provided initial results that did not make much sense. The simulation was highly unstable, and provided a temperature drop down to 50 K, which was clearly unreasonable. Additional numerical tricks had to be employed, as suggested by [12]. First of all, Adams Bashforth methods were used to improve the accuracy of the implementation, which is a strategy also backed up in general numerical reference texts [16]. Also, a time delay of needed to be added to the vaporization step in order to make the model more stable. Combined with engine data provided by other members of the Waterloo Rocketry propulsion team, the blowdown could be simulated. An rough debug image from a successful simulation using realistic parameters could be seen below.

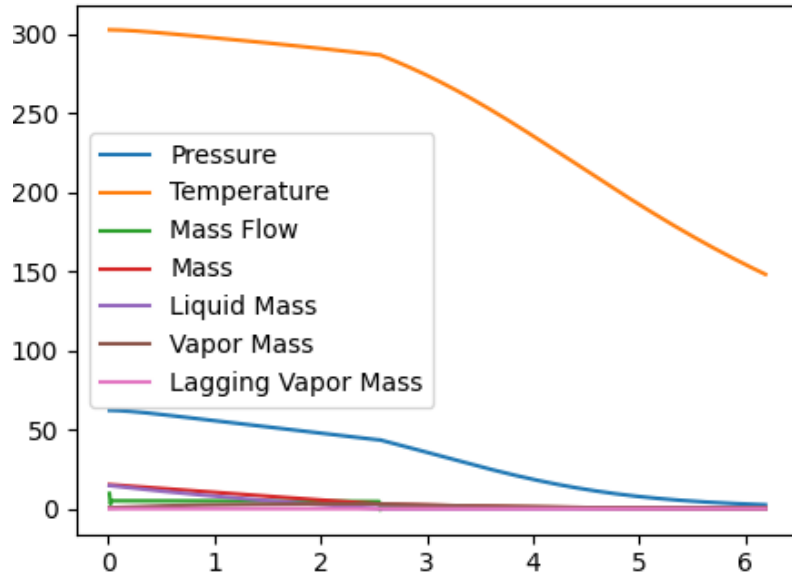


FIGURE 3: DEBUG PRINTOUT OF A BLOWDOWN SIMULATION

The second major part of the work involved the simulation of the combustion dynamics themselves. Theoretically, this would simply involve the minimization of Gibbs Free energy in a chemical kinetic system [1]. However, writing this from scratch was largely unnecessary, and an off-the-shelf solution was sought. One paid solution was RPA [17], which had been used by the team previously, and which had a built-in GUI to perform basic simulations. However, it was not fully free or open-source, and integrating it with the custom implementation presented a large amount of uncertainty. Instead, Cantera [18] was selected as the thermochemical calculator, due to its availability as a python library and the existence of seemingly comprehensive documentation for it. Basic simulations in it could then be checked against that of RPA to validate model accuracy.

The first problem encountered was that the correct propellants did not exist in the standard Cantera database. A chemical kinetics file had to be downloaded from an external source, and then modified by hand when slight difference in formatting made it incompatible with Cantera [19]. However, this enabled a fairly accurate comparison between RPA and the custom solution, when analyzed by reaction products. This can be seen in Figure 13 and Figure 14 from Appendix A. Another feature that was desirable to implement was the ability to characterize how varying percentages and sizes of aluminum could impact the performance of the engine, since it theoretically boosts performance and increases regression rate [20]. However, this was found to exceed the documented capabilities of Cantera, and was not pursued further.

The next portion of the code is the regression model of the combustion chamber. As the fuel grain combusts with the oxidizer, the surface layers regresses outwards. This in turn augments the contact surface and volume of the combustion chamber. A detailed overview of the regression and combustion dynamics are largely beyond the scope of this report. The treatment given to the subject in [21] was used heavily in preparing the implementation. The selected model for a circular grain regressing is as follows:

$$\frac{dr}{dt} = a \cdot (Total\ Fuel\ Flux)^n \cdot L^m \quad (5)$$

Where a, n, m are constants that need to be characterized for each application and which, in the words of Rick Newlands, are usually “jealously guarded” [1]. Nonetheless, [21] provides with some rough numbers that were used for the subsequent analysis. For non-circular fuel ports, a more complicated algorithm that considers the changing geometry needed to be implemented. This was done by P. Ojaghi under the direction of the author, utilizing an image processing library. A printout of a test case can be seen below.

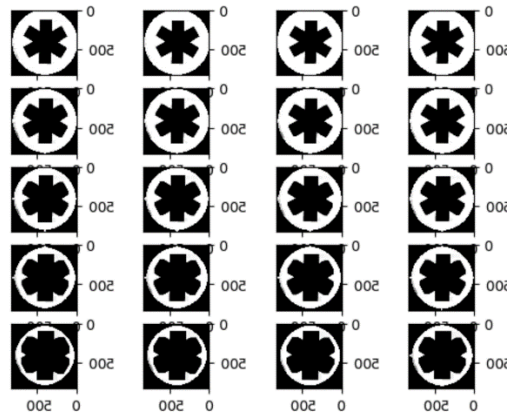


FIGURE 4: COMPLEX GEOMETRY REGRESSION MODEL (PROVIDED BY P. OJAGHI ON NOVEMBER 5, 2022)

However, this ultimately did not end up being utilized, as the selected geometry for the design cycle was circular. More complex geometries like the finocyl one in the figure above have a greater starting surface area (and thus a greater flux). However, the team chose a vortex injector for the design cycle, which injects the oxidizer at an angle to create vorticity in the flow, increasing the wall friction and thus the regression rate [22]. The best geometry to support a stable oxidizer vortex is a circular port, and thus the functionality described above did not end up being utilized.

The final portion of the code that needed to be written was the relations defining the isentropic flow through the nozzle and pressures inside the combustion chamber. [1] suggests using a pressure differential approach to correctly model the rise in the pressure of the combustion chamber, which is described by the following equation:

$$\frac{1}{m} \frac{dm}{dt} = \frac{1}{p} \frac{dp}{dt} - \frac{1}{V} \frac{dV}{dt} \quad (6)$$

The remaining equations that describe isentropic flow were described in great length in [23]. These were all applied and implemented in code. One of the difficulties that arose in implementing them was the mach number equation for determining the massflow out of the nozzle based on the area ratio. The equation was far too difficult to solve analytically, as can be seen in the figure below, taken from [23].

$$\frac{A_t}{A_y} = \frac{V_t v_y}{V_y v_t} = \left(\frac{k+1}{2} \right)^{1/(k-1)} \left(\frac{p_y}{p_1} \right)^{1/k} \sqrt{\frac{k+1}{k-1} \left[1 - \left(\frac{p_y}{p_1} \right)^{(k-1)/k} \right]} \quad (3-25)$$

FIGURE 5: EQUATION RELATING AREA RATIO AND MACH NUMBER [23]

On the other hand, the numerical solver for it behaved inconsistently. This was resolved after determining that the function was non-monotonic, which two solutions existing for each area ratio. This demonstrated well in the figure from professor Higgins [24].

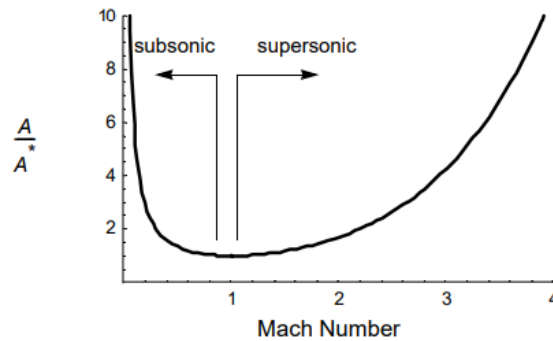


FIGURE 6: MACH NUMBER CURVE AS A FUNCTION OF AREA RATIO [24]

After these issues were resolved, the whole model was put together for final integration tests. The last remaining problem identified at this stage was the numerical instability of the code. When the step size was reduced, the peak thrust and the pressure rise time would increase dramatically. This can be seen in the two figures below, taken during development in late fall of 2022. This posed a problem as peak thrust is an important engine parameter for mechanical design, while the thrust rise time is of significant importance for flight modelling, since it determines the off-rail velocity, and thus the lower bound of the launch vehicle's stability.

Time step = 0.01

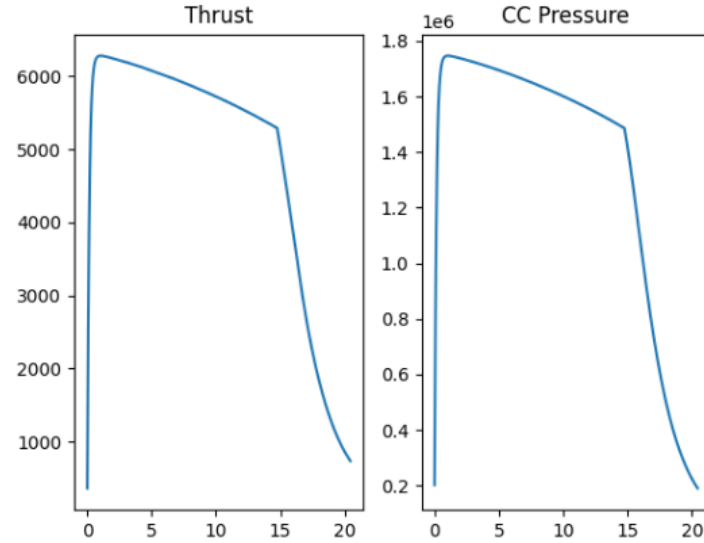


FIGURE 7: COMPLETE SIMULATION RUNNING AT A 0.01s TIME STEP

Time step = 0.05

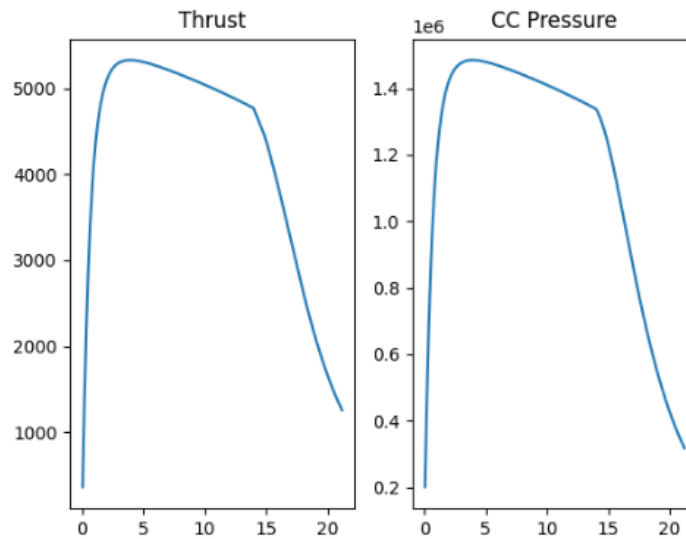


FIGURE 8: COMPLETE SIMULATION RUNNING AT A 0.05s TIME STEP

In similar numerical analysis situations, a convergence study is usually done, decreasing the step size until the results no longer significantly vary. However, the approach could not simply be used in this case due to the expensive calculation time associated with the Cantera thermochemical calculations. Simulations which ran longer than an hour were not practical to develop and debug with. To address this, an attempt was made to make an adaptive step-size algorithm. Guidance was taken from [16] on how to approach this issue. High-order integration strategies were attempted to try and gain performance without decreasing step size, but this yielded no results. The selected algorithm for an adaptive step size was as follows:

1. Make two copies of the current thermochemical object TO $\rightarrow$ (A, B)
2. Run A once at time step T
3. Run B twice at time step T/2
4. **If** A, B differ significantly:
5. T = T/2
6. Jump 1
7. **Else:**
8. Set the current thermochemical model as TO = B
9. **Continue**

This resulted in a model which successfully adjusted its step size depending on the function's gradient. However, it also suffered from significant numerical stability problems (see the figure below), and crashed whenever the liquid phase of the burn concluded, which proved to be a pervasive issue.

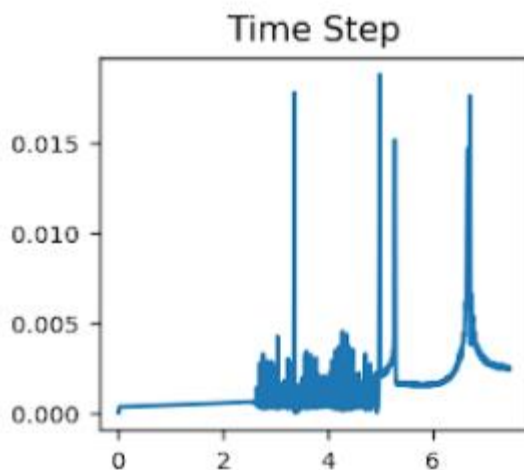


FIGURE 9: DEBUG PRINTOUT OF TIME STEP (SECONDS ON BOTH AXES) TAKEN DURING DEVELOPMENT.

Due to a lack of progress, two strategies were conceived to overcome this obstacle. They were:

1. Attempt to implement the whole simulation in a true differential solver program like Simulink
2. Use Cantera to create a 'lookup table' of thermochemical properties, which can be stored in a file, loaded into memory at runtime, and referenced via trilinear interpolation.

The second approach was attempted first, with the help of Waterloo Rocketry software member Carol Xu. A lookup generator was programmed and tested. While the generation of a significantly large and high-resolution took a massive amount of time, it did not take up a significant amount of disk space.

After running the program overnight, a sufficiently high-resolution lookup table was obtained. The programming of a trilinear interpolator (see appendix B.1) and utilization of the table finally allowed the execution of the program fast enough to significantly lower the time step. This enabled a successful convergence of the model. Running the model at 0.1 us allowed the peak thrust to converge to within 1 N, and the thrust rise time to within 1 ms (considering the 0.015s rise time 8 kN peak thrust magnitude, this was an acceptable result). Based on this, data for the burn time and peak thrust could be relayed to the engine design team, who used these parameters in designing the load-bearing mechanisms and thermal protection mechanisms of the hybrid engine.

The other approach was also attempted, for prudence. However, a derivation of an appropriate system of equations proved to be a challenge. The best-effort attempt by the author using several sources [1] [23], equations (1), (2), (3), (4), (5) and (6), plus MATLAB calculations resulted in the following:

$$\frac{dP}{dt} = P \left(\frac{1}{m} \frac{dm}{dt} - 2 \frac{dr}{dt} \frac{1}{r} \right) \quad (7)$$

$$\frac{d\omega}{dt} = C_d A \sqrt{2\rho(T_{tank}) \cdot (P_{tank}(T_{tank}) - P)} \quad (8)$$

$$\frac{df}{dt} = \frac{dr}{dt} \cdot 2\pi r L \rho_f \quad (9)$$

$$\frac{dr}{dt} = a \left(\frac{d\omega}{dt} \frac{1}{\pi r^2} \right)^n L^{m_c} \quad (10)$$

$$\frac{dm}{dt} = \frac{d\omega}{dt} + \frac{df}{dt} - A_{throat} P \cdot \text{CanteraCalc} \left(P, \frac{d\omega}{df} \right) \quad (11)$$

$$\frac{dT_{tank}}{dt} = - \frac{dm_{vapor}}{dt} \cdot H(T_{tank}) \cdot \frac{1}{m_{liquid} C_{liquid}(T_{tank})} \quad (12)$$

$$\frac{dm_{liquid}}{dt} = \frac{\rho_{vapor}}{dt} V - \frac{1}{\rho_{vapor}} \left(\rho_{liquid} \frac{m_{vapor}}{dt} + m_{vapor} \frac{\rho_{liquid}}{dt} + \rho_{liquid} m_{vapor} \frac{\rho_{vapor}}{dt} \frac{1}{\rho_{vapor}} \right) \quad (13)$$

$$\frac{dm_{vapor}}{dt} = \frac{dm_{liquid}}{dt} - \frac{d\omega}{dt} \quad (14)$$

Where P is pressure in the combustion chamber, ω is the oxidizer flow, f is the fuel flow, r is the port radius, m is the mass inside the combustion chamber, T_{tank} is the temperature of the oxidizer tank, m_{liquid} is the liquid mass in the oxidizer tank and m_{vapor} is the vapor mass in the oxidizer tank. A number of other properties like the liquid heat capacity C_{liquid} and P_{tank} are directly dependent on the temperature of the run tank, while CanteraCalc is an isentropic flow function based on the ratio of oxidizer to fuel flow and combustion chamber pressure. Equation (10) is simply equation (5) with variable names changed to avoid conflicts.

However, these equations produced only a semi-stable model in Simulink, whose solution did not match that of the python implementation. A scope capture of the result can be seen in the figure below. Further work remains in this area to get a better simulation to confirm the results of the python model.

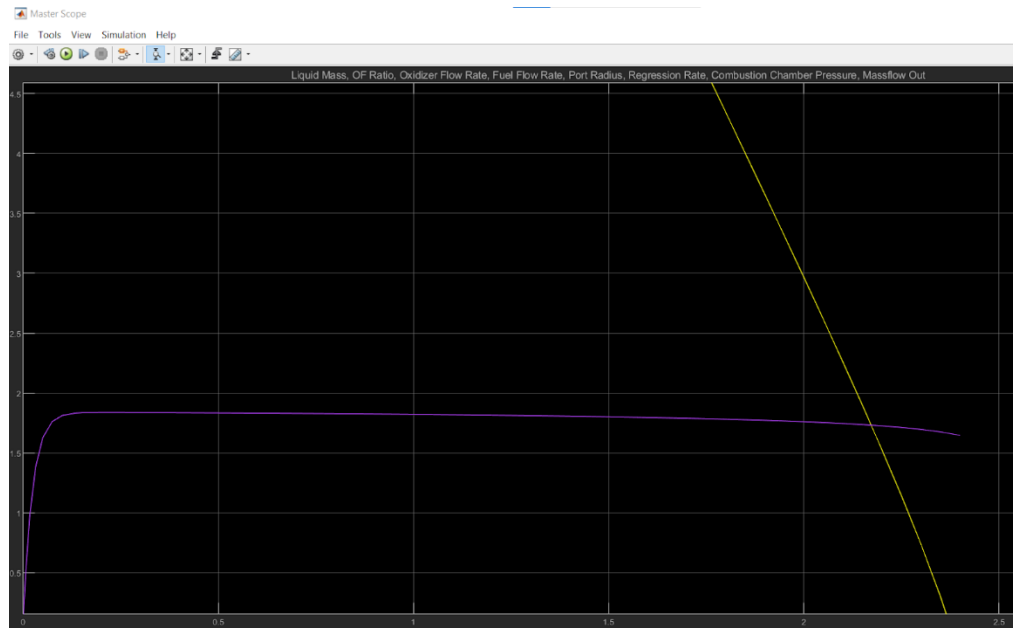


FIGURE 10: SCOPE CAPTURE OF SIMULINK SIMULATION

5.0 CONCLUSION

The project described here is still ongoing. Work remains on the simulation side. Furthermore, the engine has still not been static fired, which would serve as the final confirmation of the methods employed by the software design. Originally scheduled for the 21st of January 2023, the new tentative date is the 28th, which unfortunately puts its results outside the scope of this report.

Overall, this project demonstrated that quick and efficient methods of hybrid engine modelling can be implemented in python. It was also shown that such an implementation can aid the development efforts of a hybrid engine design campaign. The broader considerations outlined at the beginning of the report highlight the importance of efficient engine design in launch vehicle architecture, and provide and insight into the various strategies that can be used to optimize and improve a hybrid sounding rocket of a similar class.

REFERENCES

- [1] R. Newlands, *The Science and Design of the Hybrid Rocket Engine*, Morrisville, 2017.
- [2] A. Defusco, "Historical Overview of HTPB The Military's Preferred Solid Propellant Binder for a Half Century," DSIAC, 2 November 2019. [Online]. Available: <https://dsiac.org/articles/historical-overview-of-httpb-the-militarys-preferred-solid-propellant-binder-for-a-half-century/>. [Accessed 23 January 2023].
- [3] Delaney Dymont et al, "Team 139 Project Technical Report for the 2022 IREC," Waterloo Rocketry, Waterloo, ON, 2022.
- [4] McGill Rocketry Team, private communications, 2022..
- [5] Academic Space Initiative Switzerland, private communications, 2022..
- [6] C. Houlst, "Structrual Loads," 2009. [Online]. Available: <http://www.rsandt.com/reports.html#StruLoad>. [Accessed October 2022].
- [7] Metal Man Knows, "What are the Differences Between 6061 and 7075 Aluminum?," Metal Supermarkets, 23 May 2019. [Online]. Available: <https://www.metalsupermarkets.com/differences-6061-and-7075-aluminum/>. [Accessed November 2023].
- [8] F. Armao, "Aluminum Workshop: Why isn't 7075 weldable?," *The Welder*, 26 July 2018. [Online]. Available: <https://www.thefabricator.com/thewelder/article/aluminumwelding/aluminum-workshop-why-isn-t-7075-weldable-#:~:text=The%20problem%20is%20that%20while,falls%20into%20the%20unweldable%20category..> [Accessed January 2023].
- [9] OpenRocket Information Page Author, "OpenRocket: build better rockets," OpenRocket, 2022. [Online]. Available: <https://openrocket.info/>. [Accessed January 2023].
- [10] V. Goyal, "Mechanics of Composite Materials," YouTube, 17 May 2021. [Online]. Available: https://www.youtube.com/playlist?list=PLREHQnoFMshAuaShRdNQ1-tsE_r8qS6dh. [Accessed October 2022].
- [11] Ross Alexander et al, "Integrated Flight Modeling: Trajectory & Hybrid Engine Performance," July 2018. [Online]. Available: https://www.tamusrt.org/uploads/5/8/8/1/58816757/12_presentation_slides_final.pdf. [Accessed 9 November 2022].

- [12] R. Newlands, "Modelling the nitrous run tank emptying," 3 January 2012. [Online]. Available: <http://www.aspirespace.org.uk/downloads/Modelling%20the%20nitrous%20run%20tank%20emptying.pdf>. [Accessed September 2022].
- [13] A. Sotnikov, P. Ojaghi and C. Xu, "waterloo-rocketry / proPAL," Github, 22 September 2022. [Online]. Available: <https://github.com/waterloo-rocketry/ProPAL/tree/master>. [Accessed 30 November 2022].
- [14] F. White, Fluid Mechanics 8e, New York: McGraw Hill, 2016.
- [15] Y. Cengel, J. Cimbala and R. Turner, Fundamentals of Thermal-Fluid Sciences 5e, New York: McGraw Hill, 2017.
- [16] W. Press, S. Teukolsky, W. Vetterling and B. Flannery, Numerical Recipes: The Art of Scientific Computing 3e, Cambridge: Cambridge University Press, 2007.
- [17] R. S. U. Developers, "Rocket Propulsion Analysis," RP Software+Engineering UG, 2022. [Online]. Available: <https://www.rocket-propulsion.com/index.htm>. [Accessed 2 October 2022].
- [18] Cantera Developers, "Cantera is an open-source suite of tools for problems involving chemical kinetics, thermodynamics, and transport processes.," Cantera, 2022. [Online]. Available: <https://cantera.org/>. [Accessed 2 October 2022].
- [19] Mevel et al., "Mevel2015.cti," 2015. [Online]. Available: <https://shepherd.caltech.edu/EDL/PublicResources/sdt/SDToolbox/cti/>. [Accessed 23 January 2023].
- [20] Hui Tian et al, "Effect of High-Mass Fraction of Aluminum on Catalytic Hybrid Rocket Motor," *Applied Sciences*, vol. 12, no. 24, p. 13023, 2022.
- [21] R. Humble, G. Henry and W. Larson, Space Propulsion Analysis and Design (revised edition), New York: McGraw-Hill Companies Inc., 1996.
- [22] P. K. Dash and G. R. Vishal, "Study of Regression Rate in Hybrid Rockets Using Vortex Injector," in *Recent Advances in Mechanical Engineering*, Saha, UK, 2022.
- [23] G. Sutton and O. Biblarz, Rocket Propulsion Elements 9e, Hoboken, New Jersey: John Wiley & Sons, 2017.
- [24] A. Higgins, "MECH340 Fluids notes," McGill University, Montreal, 2010.

APPENDIX A: ADDITIONAL FIGURES

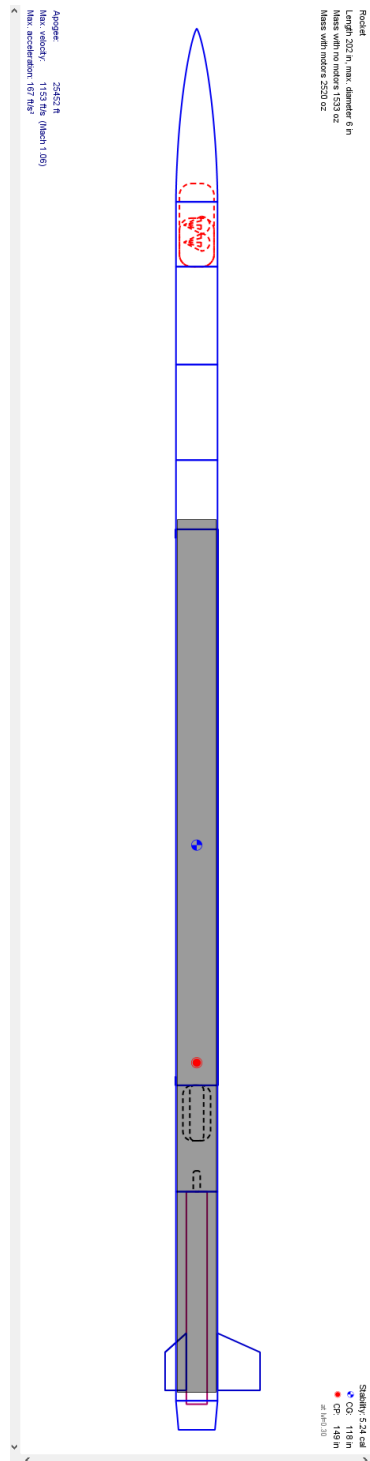


FIGURE 11: REFERENCE CASE FOR AIRFRAME DIAMETER COMPARISON: 6 INCHES

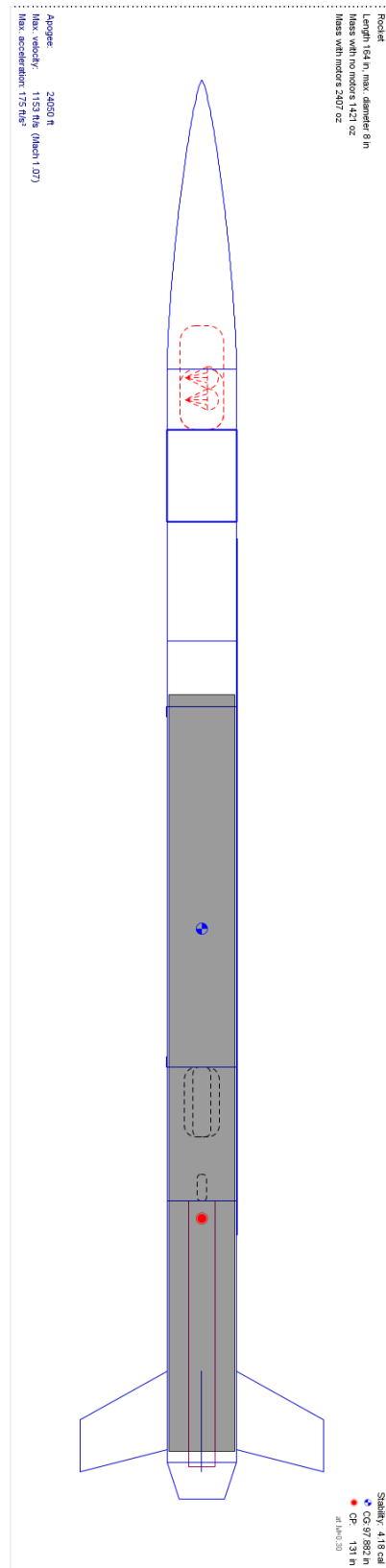


FIGURE 12: TEST CASE FOR AIRFRAME DIAMETER COMPARISON: 8 INCHES

Species	Injector mass fractions	Injector mole fractions	Nozzle inlet mass fractions	Nozzle inlet mole fractions	Nozzle throat mass fractions	Nozzle throat mole fractions	Nozzle exit mass fractions	Nozzle exit mole fractions
CO	0.3319903	0.2919832	0.3319659	0.2919701	0.3293595	0.2906482	0.3021494	0.2673139
CO2	0.0642906	0.0359872	0.0643292	0.0360099	0.0684366	0.0384374	0.1111991	0.0626137
COOH	0.0000016	0.0000009	0.0000016	0.0000009	0.0000006	0.0000003		
H	0.0003426	0.0083745	0.0003413	0.0083414	0.0001618	0.0039683	0.0000001	0.0000030
H2	0.0086435	0.1056268	0.0086455	0.1056533	0.0089021	0.1091542	0.0109717	0.1348729
H2O	0.0696272	0.0952105	0.0696286	0.0952152	0.0695603	0.0954404	0.0528532	0.0727017
HCHO,formaldehy	0.0000004	0.0000004	0.0000004	0.0000004	0.0000003	0.0000002		
HCN	0.0000043	0.0000039	0.0000043	0.0000039	0.0000025	0.0000023	0.0000002	0.0000002
HCO	0.0000066	0.0000056	0.0000066	0.0000056	0.0000025	0.0000021		
HCOOH	0.0000006	0.0000003	0.0000006	0.0000003	0.0000003	0.0000002		
HNC	0.0000007	0.0000007	0.0000007	0.0000007	0.0000003	0.0000003		
HNCO	0.0000018	0.0000010	0.0000017	0.0000010	0.0000010	0.0000006		
HNO	0.0000005	0.0000004	0.0000005	0.0000004				
HO2	0.0000002	0.0000002	0.0000002	0.0000002				
N	0.0000008	0.0000013	0.0000007	0.0000013	0.0000001	0.0000002		
N2	0.5225386	0.4595150	0.5225412	0.4595305	0.5227471	0.4612517	0.5228246	0.4624924
N2O	0.0000002	0.0000001	0.0000002	0.0000001				
NH	0.0000007	0.0000012	0.0000007	0.0000012	0.0000001	0.0000002		
NH2	0.0000011	0.0000016	0.0000011	0.0000016	0.0000003	0.0000005		
NH3	0.0000033	0.0000048	0.0000033	0.0000047	0.0000024	0.0000035	0.0000015	0.0000022
NO	0.0005969	0.0004901	0.0005916	0.0004857	0.0001594	0.0001313		
O	0.0000765	0.0001178	0.0000757	0.0001166	0.0000128	0.0000197		
O2	0.0000547	0.0000421	0.0000541	0.0000417	0.0000090	0.0000070		
OH	0.0018159	0.0026302	0.0018041	0.0026132	0.0006407	0.0009312		

FIGURE 13: RPA CALCULATION RESULTS (PROVIDED BY AARON LESZKOWIAT , OCTOBER 19, 2022)

```

In [67]: gas_new.TP = 2500, 3000000
In [68]: gas_new.equilibrate('HP')
In [69]: gas_new()

gas:
  temperature 4141.3 K
  pressure    3e+06 Pa
  density      1.9152 kg/m^3
  mean mol. weight 21.983 kg/kmol
  phase of matter gas

          1 kg          1 kmol
-----
enthalpy  5.4388e+06    1.1956e+08 J
internal energy 3.8725e+06 8.5127e+07 J
entropy    11360      2.4972e+05 J/K
Gibbs function -4.1606e+07 -9.146e+08 J
heat capacity c_p 1672    36756 J/K
heat capacity c_v 1293.8  28441 J/K

          mass frac. Y          mole frac. X          chem. pot. / RT
-----
O          0.018162          0.024954          -16.749
N2O        6.3087e-06        3.1509e-06        -43.863
N          0.00021963        0.00034468        -13.557
OH*        3.7647e-07        4.8661e-07        -27.294
H          0.0049747         0.10849          -10.544
H2         0.007918         0.086339         -21.089
O2         0.0072497         0.0049806        -33.499
OH         0.027752         0.035871         -27.294
HO2        2.8054e-05        1.8685e-05        -44.043
H2O2       1.7941e-06        1.1595e-06        -54.588
NH         4.5969e-05         6.73e-05         -24.101
NO         0.020187         0.014789         -30.306
NH2        6.791e-06         9.3169e-06        -34.645
N2O3       6.0479e-12        1.7491e-12        -77.362
HNO        1.6725e-05        1.1854e-05        -40.851
NO2        7.0818e-06        3.3839e-06        -47.056
NNH        8.8955e-06         6.7379e-06        -37.658
NH3        1.5712e-06         2.0279e-06        -45.19
N2H2       3.3878e-09         2.48e-09         -48.202
HONO       3.7004e-07         1.7302e-07        -57.6
NO3        5.996e-11         2.1258e-11        -63.805
HNO3       4.5929e-11         1.6023e-11        -74.349
N2H3       4.1918e-11         2.9688e-11        -58.746
N2H4       5.2143e-13         3.5768e-13        -69.291
NH2OH      1.8635e-09         1.2402e-09        -61.939
HNOH       4.8133e-08         3.3043e-08        -51.395
H2NO       7.1533e-08         4.9106e-08        -51.395
HNNO       1.9676e-09         9.6073e-10        -54.407
N2         0.52069          0.40859          -27.114
H2O        0.036546         0.044595         -37.838
CH4        2.0936e-10         2.8687e-10        -57.016
CH3        2.6105e-09         3.8168e-09        -46.471
T-CH2     6.2349e-09         9.7712e-09        -35.927
S-CH2     9.5117e-10         1.4906e-09        -35.927
CH         1.597e-08         2.6966e-08        -25.383
CH*        5.5667e-12         9.3993e-12        -25.383
C          7.9083e-08         1.4474e-07        -14.838
C2H3       7.7604e-14         6.3076e-14        -61.31
C2H2       7.7267e-11         6.5233e-11        -50.766
H2C2       1.5684e-12         1.3241e-12        -50.766
C2H        5.0532e-11         4.4379e-11        -40.221

```

FIGURE 14: CANTERA CALCULATION RESULTS (DONE ON OCTOBER 19TH, 2022)

APPENDIX B: CODE

B.1 TRILINEAR INTERPOLATION

```
def lookup(self, p1,p2,p3) -> tuple:

    # Manual implementation of trilinear interpolation algorithm.

    p1_val_lower = round(math.floor((p1 - self.param1_start)
/self.param1_step) * self.param1_step, 5) + self.param1_start

    p1_val_upper = round(math.ceil((p1 -
self.param1_start)/self.param1_step) * self.param1_step, 5) +
self.param1_start

    p2_val_lower = round(math.floor((p2 -
self.param2_start)/self.param2_step) * self.param2_step, 5) +
self.param2_start

    p2_val_upper = round(math.ceil((p2 -
self.param2_start)/self.param2_step) * self.param2_step +
self.param2_start, 5)

    p3_val_lower = round(math.floor((p3 -
self.param3_start)/self.param3_step) * self.param3_step, 5) +
self.param3_start

    p3_val_upper = round(math.ceil((p3 -
self.param3_start)/self.param3_step) * self.param3_step, 5) +
self.param3_start

    p1_difference = (p1 - p1_val_lower)/(p1_val_upper - p1_val_lower)
    p2_difference = (p2 - p2_val_lower)/(p2_val_upper - p2_val_lower)
    p3_difference = (p3 - p3_val_lower)/(p3_val_upper - p3_val_lower)

    C_000 = self.table[p1_val_lower, p2_val_lower, p3_val_lower]
```

```

C_100 = self.table[p1_val_upper, p2_val_lower, p3_val_lower]
C_010 = self.table[p1_val_lower, p2_val_upper, p3_val_lower]
C_001 = self.table[p1_val_lower, p2_val_lower, p3_val_upper]

C_110 = self.table[p1_val_upper, p2_val_upper, p3_val_lower]
C_011 = self.table[p1_val_lower, p2_val_upper, p3_val_upper]
C_101 = self.table[p1_val_upper, p2_val_lower, p3_val_upper]

C_111 = self.table[p1_val_upper, p2_val_upper, p3_val_upper]

C_00 = self.linear_interpolate_structs(C_000, C_100,
p1_difference)
C_10 = self.linear_interpolate_structs(C_001, C_101,
p1_difference)
C_01 = self.linear_interpolate_structs(C_010, C_110,
p1_difference)
C_11 = self.linear_interpolate_structs(C_011, C_111,
p1_difference)

C_0 = self.linear_interpolate_structs(C_00, C_01, p2_difference)
C_1 = self.linear_interpolate_structs(C_10, C_11, p2_difference)

C_final = self.linear_interpolate_structs(C_0, C_1, p3_difference)

return MockCanteraResult(*C_final)

```